МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
САМАРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Институт автоматики и информационных технологий
Кафедра «Информатика и вычислительная техника»

ОТЧЕТ

по лабораторной работе № 4

по дисциплине Программирование микропроцессоров в технических системах

на тему Прерывания в микропроцессорах

Преподаватель			26.10.27	Хмелевской А. А.
	(должность)	(подпись)	(дата)	(инициалы, фамилия)
Студенты			26.10.27	Латыпов Е. А.
	(группа)	(подпись)	(дата)	(инициалы, фамилия)
			26.10.27	Ишкачев З. П.
	(группа)	(подпись)	(дата)	(инициалы, фамилия)

Самара 2026

Цель. Изучить работу матричной клавиатуры 4×4 с Arduino Uno и реализовать определение нажатой клавиши по прерыванию с выводом её в монитор порта и звуковым подтверждением.

Суть задания

Нужно подключить матричную клавиатуру 4×4 к Arduino, ловить нажатие клавиши по прерыванию и выводить нажатую клавишу «на экран» (в вашем случае в монитор порта). Нажатие подтверждается звуковым сигналом.

Матричная клавиатура экономит пины. Вместо 16 отдельных кнопок и 16 пинов используется 4 линии строк и 4 линии столбцов, то есть 8 пинов. Кнопка стоит на пересечении строки и столбца и при нажатии их замыкает. Чтобы понять, какая именно нажата, делают сканирование: по очереди подают сигнал на каждый столбец и смотрят, на какой строке он появился. Пара «столбец и строка» однозначно определяет клавишу.

Прерывание нужно, чтобы не опрашивать клавиатуру постоянно: процессор запускает сканирование, только когда что-то нажали.

Данные

```
const int cols[4] = {4, 5, 6, 7};
const int rows[4] = {8, 9, 10, 11};
```

Пины 4–7 работают как столбцы (выходы), пины 8–11 как строки (входы). Пины 8–11 выбраны не случайно: они лежат на одном порту (порт B), и одно прерывание по изменению состояния следит сразу за всеми четырьмя.

```
const char keys[4][4] = {{'1','2','3','a'},
{'4','5','6','b'}, {'7','8','9','c'},
{'*','0','#','d'}};
```

Таблица символов в том виде, как они напечатаны на клавиатуре.

```
volatile bool pressed = false;
volatile unsigned long last = 0;
```

`pressed` это флаг «было нажатие», он ставится в прерывании и читается в `loop()`. `last` это время последнего принятого срабатывания, нужно для антидребезга. Обе переменные `volatile`, потому что меняются внутри прерывания.

Обработчик прерывания

```
ISR(PCINT0_vect) {
  if (millis() - last >= 50) {
    last = millis();
    pressed = true;
  }
}
```

`PCINT0_vect` это вектор прерывания по изменению состояния любого из пинов 8–13. Обработчик короткий: проверяет, что с прошлого срабатывания прошло не меньше 50 мс (иначе это дребезг контактов), и ставит флаг. Он срабатывает и при нажатии, и при отпускании кнопки.

`setup()`

```
for (int i = 0; i < 4; i++) {
  digitalWrite(cols[i], LOW);
  pinMode(cols[i], OUTPUT);
  pinMode(rows[i], INPUT_PULLUP);
}
```

Столбцы настраиваются как выходы и сразу получают LOW, поэтому в покое все они «прижаты к земле». Сначала записывается LOW, и только потом включается режим выхода, чтобы на пине не было короткого скачка. Строки настраиваются как входы с внутренней подтяжкой к питанию: без нажатия на них HIGH. Когда кнопка нажата, она соединяет строку с землёй (через столбец, прижатый к LOW), и на строке появляется LOW. Благодаря этому не нужны ни внешние резисторы, ни диоды.

```
PCICR |= (1 << PCIE0);
PCMSK0 |= 0x0F;
```

Первая строка включает группу прерываний по изменению состояния для порта B (PCIE0). Вторая выбирает, какие пины внутри группы следят за изменением: `0x0F` это двоичное `00001111`, то есть пины 8, 9, 10, 11 (нижние четыре бита).

`loop()`: определение клавиши

```
if (!pressed) return;
delay(20);
```

Пока флага нет, ничего не происходит. Если нажатие было, сначала пауза 20 мс, чтобы контакты успокоились и сигнал стал устойчивым.

```
PCICR &= ~(1 << PCIE0);
```

Прерывание выключается на время сканирования. Иначе само сканирование (оно переключает столбцы и меняет состояние строк) вызывало бы новые срабатывания.

```
for (int c = 0; c < 4; c++) {
  for (int i = 0; i < 4; i++) pinMode(cols[i], i == c ?
OUTPUT : INPUT);
```

Сканирование по столбцам. На каждом шаге только один столбец `c` остаётся выходом с уровнем LOW, а остальные переключаются в режим входа, то есть «отключаются». Так нажатые клавиши в других столбцах не могут повлиять на результат, и пины не замыкаются между собой.

```
  for (int r = 0; r < 4; r++) {
    if (digitalRead(rows[r]) == LOW) {
```

Для текущего столбца проверяются все четыре строки. Если на какой-то из них LOW, значит, нажата клавиша на пересечении этого столбца и этой строки.

```
Serial.println(keys[3-c][3-r]);
tone(12, 1000, 100);
```

Нажатая клавиша печатается в монитор порта, а на пин 12 подаётся звук частотой 1000 Гц длительностью 100 мс (`tone` не блокирует программу). Индексы `[3-c][3-r]` нужны потому, что шлейф подключён к плате «перевёрнуто»: строки и столбцы поменялись местами и идут в обратном порядке. Эта формула разворачивает их обратно, и таблицу `keys` можно записать так, как она напечатана на клавиатуре.

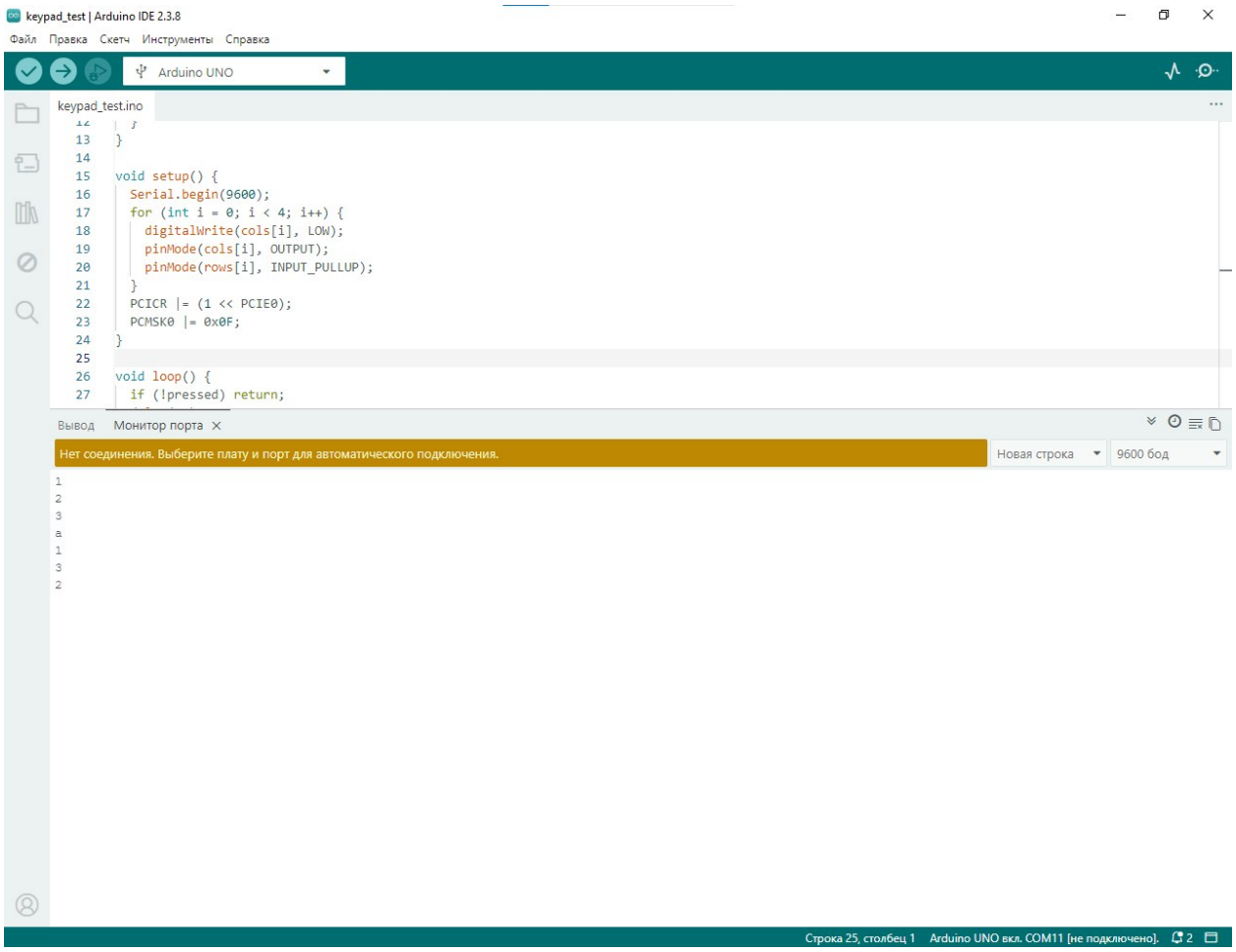
```
delay(50);
while (digitalRead(rows[r]) == LOW);
delay(50);
```

Программа ждёт, пока клавишу отпустят, и ещё 50 мс, пока погаснет дребезг при отпускании. Благодаря этому одно нажатие печатается один раз, даже если держать клавишу.

Завершение

```
for (int i = 0; i < 4; i++) pinMode(cols[i], OUTPUT);
pressed = false;
PCIFR |= (1 << PCIF0);
PCICR |= (1 << PCIE0);
```

Все столбцы возвращаются в режим выхода с LOW (исходное состояние), флаг сбрасывается, затем сбрасывается накопившийся флаг прерывания `PCIF0` (в этом регистре сброс делается записью единицы, как ни странно) и прерывание включается обратно. Без сброса накопленный флаг сразу вызвал бы лишнее срабатывание.



Вывод. В ходе работы подключена матричная клавиатура 4×4 (столбцы к пинам 4–7, строки к пинам 8–11) и написана программа, определяющая нажатую клавишу. Нажатие фиксируется прерыванием по изменению состояния пинов (`PCINT0`), а сама клавиша находится сканированием столбцов в основном цикле. Это позволяет не опрашивать клавиатуру постоянно, а работать только по событию. Дребезг контактов подавляется программно через `millis()` и паузы, а использование внутренних подтяжек `INPUT_PULLUP` позволило обойтись без внешних резисторов и диодов. Программа корректно определяет все 16 клавиш, выводит символ в монитор порта и подтверждает нажатие звуковым сигналом.